MRP-int

Qianyu Dong

2025-12-09

Contents

Data Overview

1. Population data `acs_pop`.
2. One set of PS `ps_df` and NPS `nps_df` by `source("code/tests/create_test_sample.R")`

```
##   Samples regenerated
```

```
data.frame(
  Sample = c("Population", "PS", "NPS"),
  N = c(nrow(acs_pop), nrow(ps_df), nrow(nps_df)),
  HICOV_Rate = c(mean(acs_pop$PUBCOV), mean(ps_df$PUBCOV), mean(nps_df$PUBCOV))
)
```

```
##       Sample       N HICOV_Rate
## 1 Population 1475631  0.6030925
## 2         PS    7307  0.6194060
## 3        NPS   72848  0.4360861
```

Model Specification

The idea of this integrated MRP (MRP-INT) is to use the estimated inclusion probabilities as predictors in the regression models.

Inclusion probability following Valliant 2019

Let S_i be the indicator whether the i is in PS or NPS by

$$S_i = \begin{cases} 1, & \text{if unit } i \text{ is in the nonprobability sample,} \\ 0, & \text{if unit } i \text{ is in the probability sample.} \end{cases}$$

Assign a weight of 1 to each nonprobability case and fit a weighted binary regression to predict the probability of being in the nonprobability sample. Valliant 2019 pointed that a weight adjustment is needed when the NPS size is not a negligible to the whole population. The weight adjustment is on PS such that sum of the NPS weight (i.e. size of the NPS) and adjusted PS weight equal to original total PS weight.

Specifically, let w_i denote the original weight of element i in the PS, let

$$\hat{N} = \sum_{i \in PS} w_i$$

be the estimated population size from the reference sample, and let n be the sample size of the NPS. The adjusted weight for each PS unit is

$$w_i^* = w_i \frac{\hat{N} - n}{\hat{N}}.$$

Let $\mathbf{x}_i$ denote the covariate vector used in the weighted binary regression model (the same sets are used as MR later). Here both PS and NPS are used in this regression, and the likelihood is weighted by survey weight.

$$S_i \mid \mathbf{x}_i \sim \text{Bernoulli}(\psi_i), \tag{1}$$
$$\text{logit}(\psi_i) = \gamma_0 + \mathbf{x}_i^\top \gamma, \tag{2}$$

Then the inclusion probabilities are obtained by

$$\psi_i = \Pr(S_i = 1 \mid \mathbf{x}_i) = \text{expit}(\gamma_0 + \mathbf{x}_i^\top \gamma), \qquad \psi_i \in (0, 1).$$

```
sel_dat <- rbind(
    cbind(sel_MR, PWGTP = 1),
    sel_ps
  )

  if(adjust==TRUE){
  ## 1) Estimated population size from PS
  N_hat <- sum(sel_ps$PWGTP)
  ## 2) Size of nonprobability sample
  n_np <- nrow(sel_MR)
  ## 3) Adjustment factor
  adj_factor <- (N_hat - n_np) / N_hat
  ## 4) Adjust PS weights
```

```r
sel_ps$PWGTP <- sel_ps$PWGTP * adj_factor
sel_MR$PWGTP=1
sel_dat <- rbind(
  sel_MR[, c("AGEP_binned","SEX","RAC1P","PUMA","S","PWGTP")],
  sel_ps[, c("AGEP_binned","SEX","RAC1P","PUMA","S","PWGTP")]
)
}else{
  adj_factor=NULL
}

sel_fit <- stats::glm(
  S ~ AGEP_binned + SEX + RAC1P,
  data = sel_dat,
  family = binomial(),
  weights = sel_dat$PWGTP
)
```

We get two sets of estimated inclusion probabilities. The first is continuous ψ_i to use as linear predictor in the outcome model. Then we need discrete inclusion categories Ψ_i by rounding with one digit, such that observations within the same inclusion probabilities bin are assigned to the same group and share the same intercept.

```r
#bin_val is 0.11, 0.12, etc, psi is the predicted psi, using bin_val represents the random-effect gro
bin_fun <- function(p) round(p, 1)  #round(p, 1)  may end up only has two groups if inclusion probs a
predict_psi <- function(df) {
  if (is.null(df) || nrow(df) == 0) return(list(psi = numeric(0), psi_bin = integer(0)))
  df2 <- align_levels(df)
  p <- stats::predict(sel_fit, newdata = df2, type = "response")
  p <- pmin(pmax(p, psi_eps), 1 - psi_eps)#to avoid 0
  list(psi = as.numeric(p), bin_val = bin_fun(p))
}

psi_train <- predict_psi(nps_ca)
psi_ps    <- predict_psi(ps)
psi_pop   <- predict_psi(acs_pop)

# Build a consistent bin map across ALL datasets
all_bin_vals <- unique(c(psi_train$bin_val, psi_ps$bin_val, psi_pop$bin_val))
all_bin_vals <- sort(unique(all_bin_vals))
bin_map <- setNames(seq_along(all_bin_vals), all_bin_vals)

map_bins <- function(bin_val_vec) as.integer(bin_map[as.character(bin_val_vec)])

psi_bin_train <- map_bins(psi_train$bin_val)
psi_bin_ps    <- map_bins(psi_ps$bin_val)
psi_bin_pop   <- map_bins(psi_pop$bin_val)
G <- length(all_bin_vals)
```

Multilevel regression

Let Y_i be the binary outcome of insurance coverage for unit i in the nonprobability sample. Let $\mathbf{x}_i$ denote the covariates, and let ψ_i and Ψ_i be the continuous inclusion probability and its discretized category as defined

above. The MRP-INT outcome model is

$$Y_i \mid \mathbf{x}_i, \psi_i, \Psi_i \sim \text{Bernoulli}(p_i), \tag{3}$$
$$\text{logit}(p_i) = \mathbf{x}_i^\top \beta + \beta_\psi \, \text{logit}(\psi_i) + \zeta_{\Psi_i}. \tag{4}$$
$$\beta \sim \text{Normal}(0, \sigma_\beta) \tag{5}$$
$$\beta_\psi \sim \text{Normal}(0, \sigma_\psi) \tag{6}$$
$$\zeta_g \mid \sigma_\psi \sim \mathcal{N}(0, \sigma_\psi^2), \qquad g = 1, \ldots, G, \tag{7}$$
$$\sigma_\psi \sim \text{Exponential}(\lambda), \tag{8}$$

Here β are fixed-effect coefficients for the covariates $\mathbf{x}_i$, β_ψ is the coefficient on the continuous inclusion term $\text{logit}(\psi_i)$, and ζ_{Ψ_i} is a random intercept associated with the inclusion bin Ψ_i. The inclusion bin random effects follow Exponential distribution where λ is a rate parameter from data such that $\lambda = 1/\hat{\sigma}_Y$. Moderately informative priors are used for covariates, in which $\sigma_\beta = 2.5 * sd(Y)/sd(X)$).

```
functions {
  real bern_ll_slice(array[] int y_slice, int start, int end,
                     matrix X,
                     array[] int psi_bin,
                     vector lp_psi,
                     vector beta,
                     real beta_psi,
                     vector zeta) {
    int M = end - start + 1;
    vector[M] alpha_slice = beta_psi * lp_psi[start:end] + zeta[psi_bin[start:end]];
    return bernoulli_logit_glm_lpmf(y_slice | X[start:end, ], alpha_slice, beta);
  }
}
data {
  // --- Training data
  int<lower=1> n;
  int<lower=1> k;
  matrix[n, k] X;
  array[n] int<lower=0, upper=1> y;

  // inclusion prob & bin for MRP-INT
  // vector<lower=0, upper=1>[n] psi;          // estimated inclusion probabilities for training units
  int<lower=1> G;
  vector[n] lp_psi;
  // number of psi bins
  array[n] int<lower=1, upper=G> psi_bin;   // bin index for each training unit

  // reduce_sum grainsize
  int<lower=1> grainsize;
  // ---prior
  vector[k] beta_scale;      // Vector of scales for beta
  real beta_psi_scale;       // Scale for beta_psi
  real sigma_psi_rate;       // Rate for sigma_psi
}


parameters {
  vector[k] beta;
  real beta_psi;                    // fixed effect for logit(psi)
```

```
  // psi-bin random effects
real<lower=0, upper=5> sigma_psi;
  vector[G]     zeta_raw;
}

transformed parameters {
  vector[G] zeta = sigma_psi * zeta_raw;    // psi-bin RE
}



model {
  // Data-informed priors
  beta      ~ normal(0, beta_scale);
  beta_psi  ~ normal(0, beta_psi_scale);
  sigma_psi ~ exponential(sigma_psi_rate);
  zeta_raw  ~ normal(0, 1);

  // Parallelized likelihood
  target += reduce_sum(bern_ll_slice, y, grainsize,
                       X, psi_bin, lp_psi, beta, beta_psi, zeta);
}
```

Poststratification

For posterior samples, let $\beta^{(s)}$, $\beta_\psi^{(s)}$, and $\zeta_g^{(s)}$ denote the s-th posterior draw of β, β_ψ, and ζ_g, $s = 1, \dots, S$.

Si 2023 assumed known population, so the cell are build using population data. Let $c = 1, \dots, C$ index population cells defined by combinations of covariates and PUMA. Let

- $\mathbf{x}_c$ be the cell-level covariate vector.
- ψ_c be the inclusion probability obtained by applying the selection model to the cell-level covariate vector.
- $\Psi_c = b(\psi_c)$ be the corresponding inclusion bin.
- w_c be the population weight from summing each individual weight, which is 1 in population data.

For each posterior draw s, the cell-level linear predictor and probability are

$$\eta_c^{(s)} = \mathbf{x}_c^\top \beta^{(s)} + \beta_\psi^{(s)} \operatorname{logit}(\psi_c) + \zeta_{\Psi_c}^{(s)}, \tag{9}$$

$$p_c^{(s)} = \operatorname{expit}(\eta_c^{(s)}). \tag{10}$$

The PUMA-level estimate for area j is then

$$\theta_j^{(s)} = \frac{\sum_{c \in j} w_c \, p_c^{(s)}}{\sum_{c \in j} w_c}.$$

```r
# ---------- 1) individuals to cells ----------

collapse_to_cells_int <- function(df, train_ref, PUMA_lev,
                                  psi_vec,
                                  psi_bin_vec,
                                  design_formula = ~ 0 + AGEP_binned + SEX + RAC1P,
                                  weight_col = NULL) {
  # Align to training factors
  df <- .align_to_train(df, train_ref, PUMA_lev)

  # Weights
  w <- if (!is.null(weight_col) && weight_col %in% names(df)) df[[weight_col]] else 1
  w <- as.numeric(w)
  w[!is.finite(w) | w < 0] <- 0

  # Attach psi and psi_bin to rows
  if (length(psi_vec) != nrow(df) || length(psi_bin_vec) != nrow(df)) {
    stop("psi_vec and psi_bin_vec must have length nrow(df)")
  }
  df$psi     <- as.numeric(psi_vec)
  df$psi_bin <- as.integer(psi_bin_vec)


  cells <- df |>
    dplyr::mutate(W = w) |>
    dplyr::group_by(AGEP_binned, SEX, RAC1P, PUMA) |>
    dplyr::summarise(
      w       = sum(W),
      psi     = mean(psi),
      .groups = "drop"
    )
  cells$psi_bin<-bin_fun(cells$psi)

  # Design matrix for outcome model
  Xp <- model.matrix(design_formula, data = cells)
  g  <- as.integer(cells$PUMA)  # group index 1..J

  list(
    Xp    = Xp,
    g     = g,
    w     = cells$w,
    psi   = cells$psi,
    psi_bin = as.integer(cells$psi_bin),
    cells = cells
  )
}
# ---------- 2) extract CmdStanR draws
get_params_int_draws <- function(fit) {
  # beta: S x K
  beta_mat <- fit$draws("beta", format = "matrix")

  # beta_psi: S-length vector
  beta_psi_mat <- fit$draws("beta_psi", format = "matrix")
```

```
  beta_psi_vec <- as.numeric(beta_psi_mat[, 1])

  # zeta: S x G   (columns like zeta[1], zeta[2], ...)
  zeta_mat <- fit$draws("zeta", format = "matrix")

  list(
    beta     = beta_mat,
    beta_psi = beta_psi_vec,
    zeta     = zeta_mat
  )
}

# ---------- 3) poststrat by cell  ----------

poststrat_int_SxJ <- function(beta,
                              beta_psi,
                              zeta,
                              Xp,
                              g,
                              psi,
                              psi_bin,
                              w = NULL,
                              J = max(g)) {
  N <- nrow(Xp)
  S <- nrow(beta)

  if (length(psi) != N) stop("psi must have length N = nrow(Xp)")
  if (length(psi_bin) != N) stop("psi_bin must have length N = nrow(Xp)")
  if (length(beta_psi) != S) stop("beta_psi must have length S = nrow(beta)")

  # weights
  if (is.null(w)) w <- rep(1, N) else w <- as.numeric(w)
  w[!is.finite(w) | w < 0] <- 0

  # logit(psi) for cells
  lp_psi <- qlogis(psi)

  # Fixed-effects part: N x S
  eta_fixed <- Xp %*% t(beta)   # (N x K) %*% (K x S) = N x S

  # Selection term: beta_psi_s * lp_psi_i
  eta_sel <- outer(lp_psi, beta_psi, "*")  # N x S

  # Psi-bin random effects: zeta_s,psi_bin[i]
  # zeta is S x G; transpose to G x S and index by psi_bin
  zeta_t   <- t(zeta)                              # G x S
  eta_psiRE <- zeta_t[psi_bin, , drop = FALSE]     # N x S

  # Total linear predictor and probabilities
  eta <- eta_fixed + eta_sel + eta_psiRE
  p   <- plogis(eta)  # N x S

  # Split indices by PUMA
```

```
  idx_by_j <- split(seq_len(N), factor(g, levels = seq_len(J)))

  # get weights within PUMA
  w_norm <- lapply(idx_by_j, function(idx) {
    if (length(idx) == 0) return(numeric(0))
    sw <- sum(w[idx])
    if (sw > 0) w[idx] / sw else rep(0, length(idx))
  })

  # Aggregate to PUMA
  mu <- matrix(NA_real_, nrow = S, ncol = J)
  for (j in seq_len(J)) {
    idx <- idx_by_j[[j]]
    if (length(idx) == 0) next
    wj <- w_norm[[j]]
    # p[idx, ] is (length(idx) x S); crossprod gives S x 1
    mu[, j] <- as.numeric(crossprod(wj, p[idx, , drop = FALSE]))
  }

  mu
}
```

Fit Model

```
# Profile getMRP function
cat("   Profiling getMRP()...\n")
```

```
##    Profiling getMRP()...
```

```
start_time <- Sys.time()


# use ps$weights for PWGTP
# ps_df$PWGTP=ps$weights

tryCatch(
  {

    mrp_int <- getMRP_INT(
      MR = nps_df,
      ps = ps_df,
      acs_pop = acs_pop,
      mod = mod,
      adjust=TRUE
    )

    mrp_time <- Sys.time() - start_time
    cat("  mrp_int completed in", round(mrp_time, 2), attr(mrp_time, "units"), "\n")

    # Post Stratification results (MRP-R & MRP-P)
```

```
    mrpra <- mrp_int$puma_summary_mrpp



    cat("- MRP-R results:", nrow(mrpra), "PUMAs\n")

  error = function(e) {
    cat("  getMRP failed:", e$message, "\n")
  }
  }
)
```

```
## Running MCMC with 2 parallel chains, with 4 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 1500 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 1500 [  0%]  (Warmup)
## Chain 1 Iteration:  100 / 1500 [  6%]  (Warmup)
## Chain 2 Iteration:  100 / 1500 [  6%]  (Warmup)
## Chain 2 Iteration:  200 / 1500 [ 13%]  (Warmup)
## Chain 1 Iteration:  200 / 1500 [ 13%]  (Warmup)
## Chain 2 Iteration:  300 / 1500 [ 20%]  (Warmup)
## Chain 1 Iteration:  300 / 1500 [ 20%]  (Warmup)
## Chain 2 Iteration:  400 / 1500 [ 26%]  (Warmup)
## Chain 1 Iteration:  400 / 1500 [ 26%]  (Warmup)
## Chain 2 Iteration:  500 / 1500 [ 33%]  (Warmup)
## Chain 2 Iteration:  501 / 1500 [ 33%]  (Sampling)
## Chain 1 Iteration:  500 / 1500 [ 33%]  (Warmup)
## Chain 1 Iteration:  501 / 1500 [ 33%]  (Sampling)
## Chain 2 Iteration:  600 / 1500 [ 40%]  (Sampling)
## Chain 1 Iteration:  600 / 1500 [ 40%]  (Sampling)
## Chain 2 Iteration:  700 / 1500 [ 46%]  (Sampling)
## Chain 1 Iteration:  700 / 1500 [ 46%]  (Sampling)
## Chain 2 Iteration:  800 / 1500 [ 53%]  (Sampling)
## Chain 2 Iteration:  900 / 1500 [ 60%]  (Sampling)
## Chain 1 Iteration:  800 / 1500 [ 53%]  (Sampling)
## Chain 2 Iteration: 1000 / 1500 [ 66%]  (Sampling)
## Chain 1 Iteration:  900 / 1500 [ 60%]  (Sampling)
## Chain 2 Iteration: 1100 / 1500 [ 73%]  (Sampling)
## Chain 1 Iteration: 1000 / 1500 [ 66%]  (Sampling)
## Chain 2 Iteration: 1200 / 1500 [ 80%]  (Sampling)
## Chain 1 Iteration: 1100 / 1500 [ 73%]  (Sampling)
## Chain 2 Iteration: 1300 / 1500 [ 86%]  (Sampling)
## Chain 2 Iteration: 1400 / 1500 [ 93%]  (Sampling)
## Chain 1 Iteration: 1200 / 1500 [ 80%]  (Sampling)
## Chain 2 Iteration: 1500 / 1500 [100%]  (Sampling)
## Chain 2 finished in 425.0 seconds.
## Chain 1 Iteration: 1300 / 1500 [ 86%]  (Sampling)
## Chain 1 Iteration: 1400 / 1500 [ 93%]  (Sampling)
## Chain 1 Iteration: 1500 / 1500 [100%]  (Sampling)
## Chain 1 finished in 487.2 seconds.
##
## Both chains finished successfully.
## Mean chain execution time: 456.1 seconds.
```

```
## Total execution time: 487.3 seconds.

##   mrp_int completed in 8.18 mins
## - MRP-R results: 281 PUMAs
```

Results

Inlcusion probability bins

```
unique(mrp_int$psi_train$bin_val)
```

```
## [1] 0.7 0.5 0.8 0.3 0.2
```

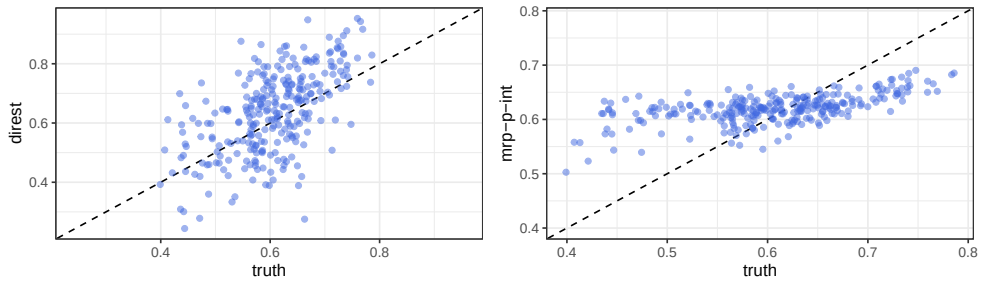
Convergence diagnostic (rhat)

Close to 1 suggests good convergence.

```
max(mrp_int$rhat)
```

```
## [1] 1.005569
```

PUMA level estimates



Metrics

```
## # A tibble: 2 x 5
##   model         MSE    MAB Coverage `Int. Score`
##   <chr>       <dbl>  <dbl>    <dbl>        <dbl>
## 1 direst    0.0138  0.0954   0.868         0.669
## 2 mrp-p-int 0.00475 0.0541   0.0427        1.99
```